

Programming Project Solutions

Link to `ipynb` file: https://github.com/Adampi210/Purdue-ECE-64700-Spring-2025-Programs/blob/main/projects/project01/project_01_sol.ipynb

Problem 1

(a)

The function is:

$$f(x_1, x_2) = x_1^2 + 3x_1x_2 + 9x_2^2 + 2x_1 - 5x_2$$

The plot is below.

In [170...

```
import matplotlib.pyplot as plt
import numpy as np

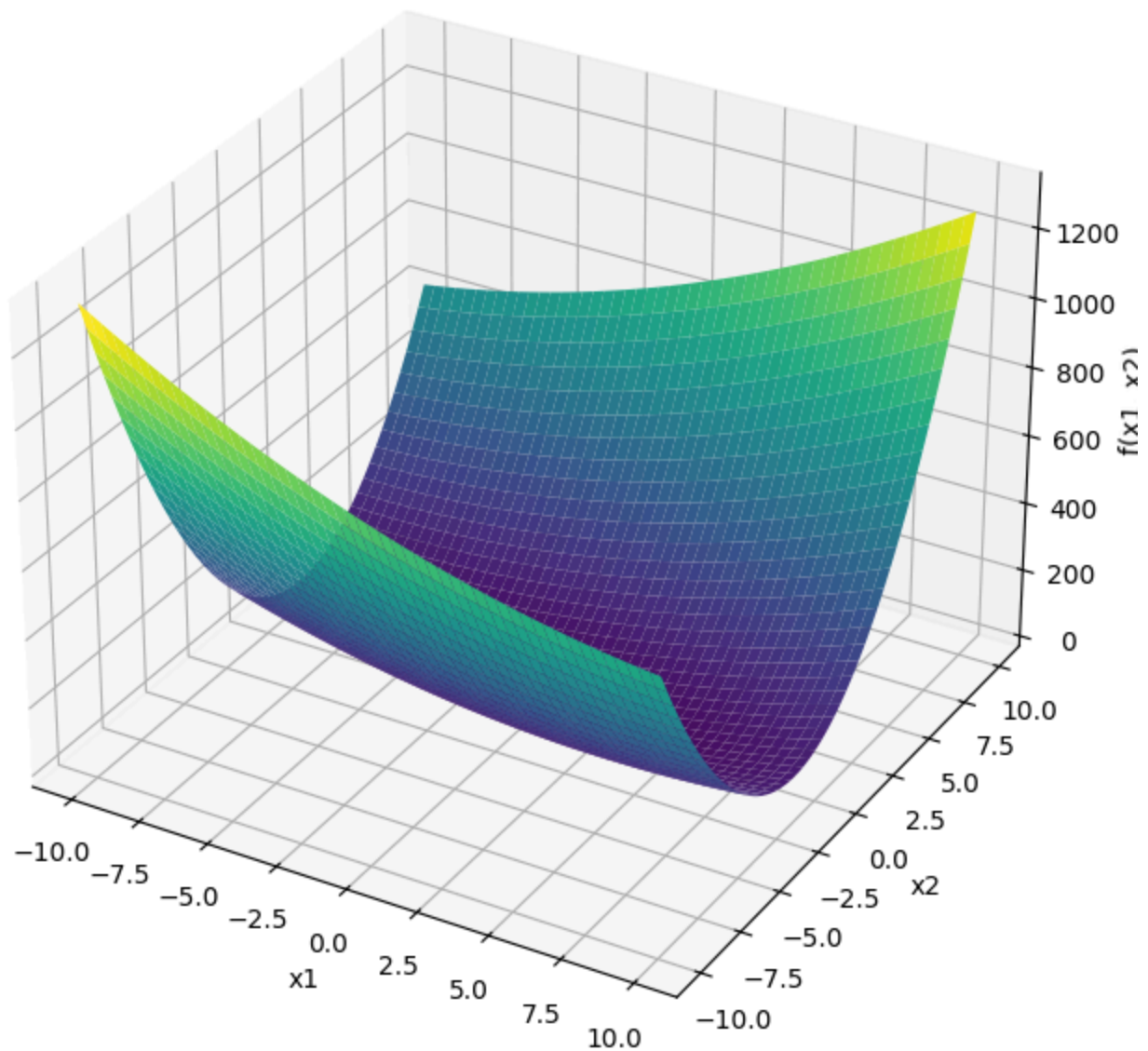
# NOTE: Use "Run ALL" button to get this seed
np.random.seed(0)

def f(x1, x2):
    return x1**2 + 3 * x1 * x2 + 9 * x2**2 + 2 * x1 - 5 * x2

# Set the limits and get z values using meshgrid
lims = (-10, 10)
x1 = np.linspace(lims[0], lims[1], 1000)
x2 = np.linspace(lims[0], lims[1], 1000)
X1, X2 = np.meshgrid(x1, x2)
Z = f(X1, X2)

# Plot
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection="3d")
surf = ax.plot_surface(X1, X2, Z, cmap="viridis")
ax.set_xlabel("x1")
ax.set_ylabel("x2")
ax.set_zlabel("f(x1, x2)")
ax.set_title("3D Picture of the Function f(x1, x2)")
plt.show()
```

3D Picture of the Function $f(x_1, x_2)$



From the plot above, we it appears the function is convex. Below are the curves on the lines going through the point $(0, 0)$ in random directions.

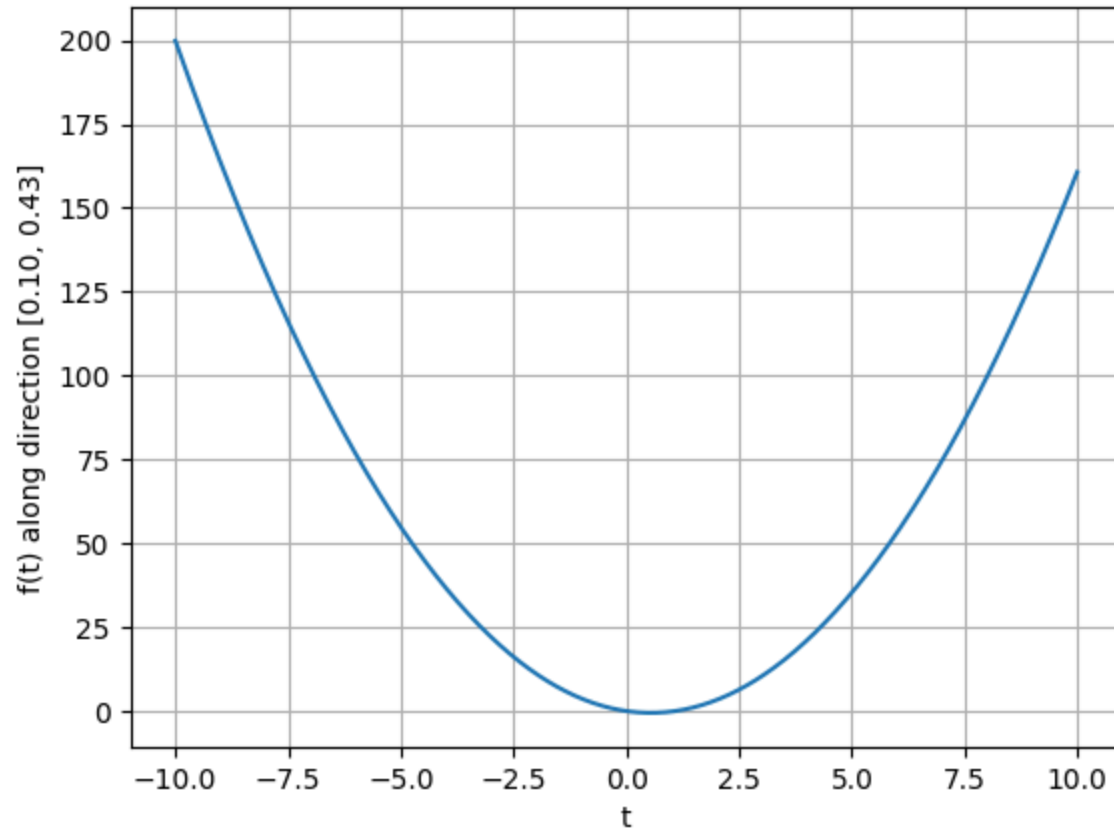
In [171...

```
# Plot the function in 2D along a direction that goes through  $(0, 0)$ 
def dir_f(t, dir_vector):
    x1 = t * dir_vector[0]
    x2 = t * dir_vector[1]
    return f(x1, x2)

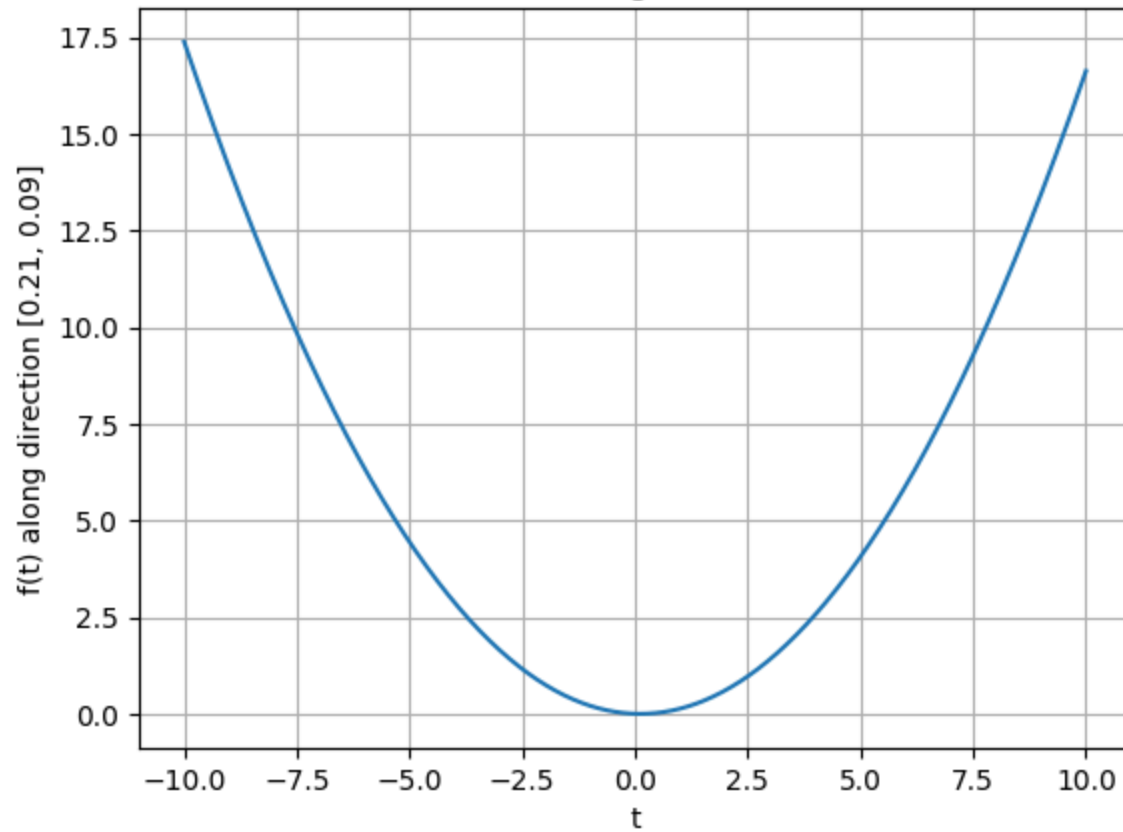
# t for plotting and random directions
lims = (-10, 10)
T = np.linspace(lims[0], lims[1], 1000)
directions = np.random.uniform(-1, 1, (5, 2))
directions[-1] = [1, 0]
# Plot along directions
for d in directions:
    f_2d_values = [dir_f(t, d) for t in T]
    plt.plot(T, f_2d_values)
    plt.xlabel("t")
    plt.ylabel(f"f(t) along direction [{d[0]:.2f}, {d[1]:.2f}]")
    plt.title(f"Function f(x1, x2) along Direction [{d[0]:.2f}, {d[1]:.2f}]")
```

```
plt.grid(True)  
plt.show()  
plt.close()
```

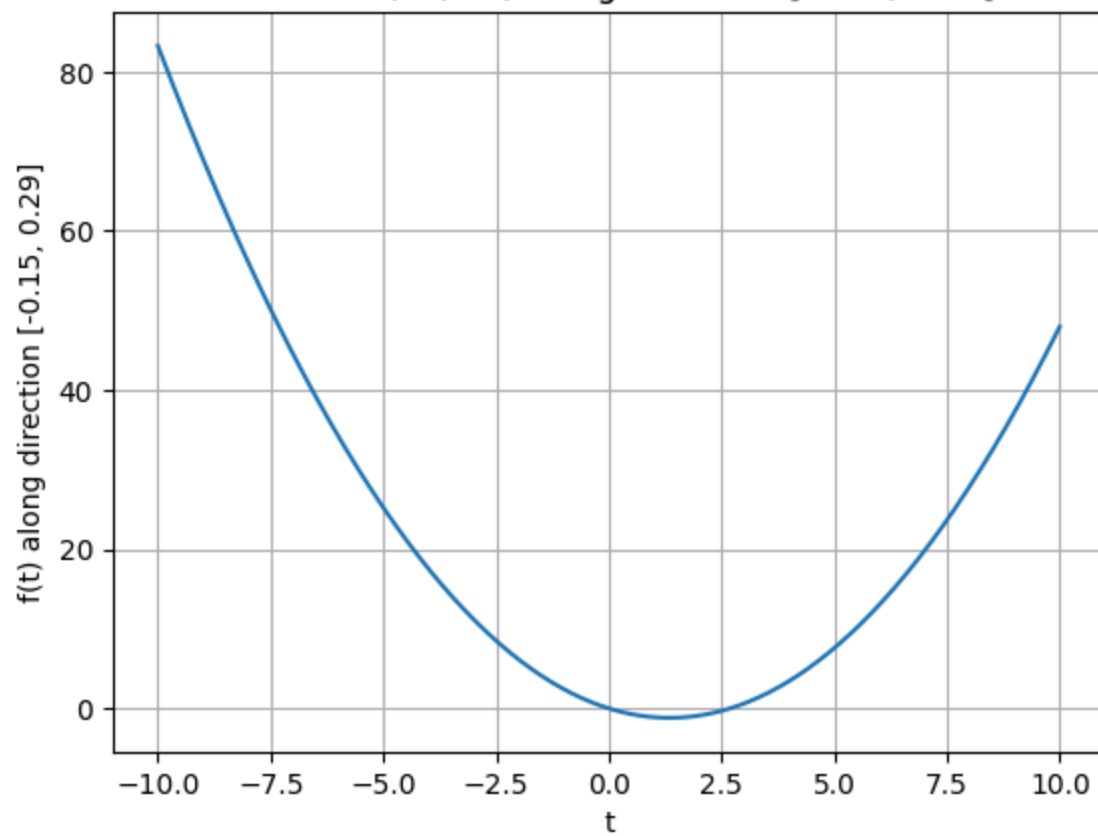
Function $f(x_1, x_2)$ along Direction $[0.10, 0.43]$



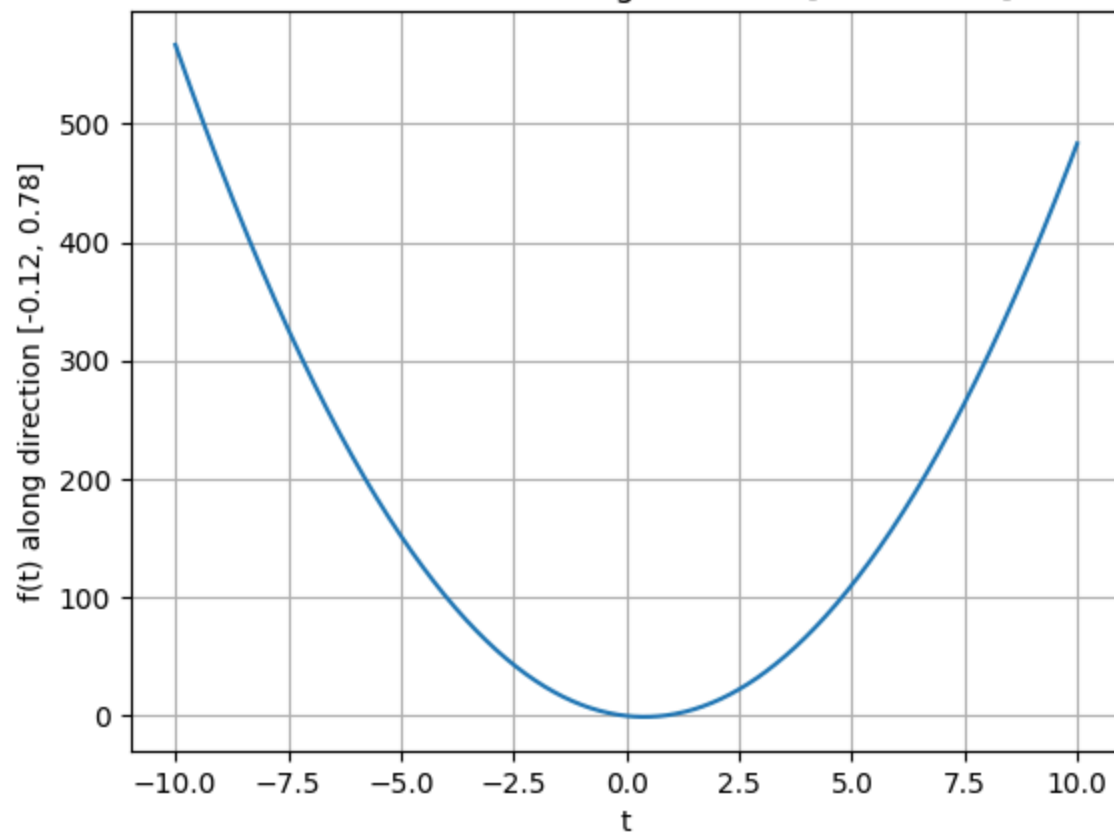
Function $f(x_1, x_2)$ along Direction $[0.21, 0.09]$

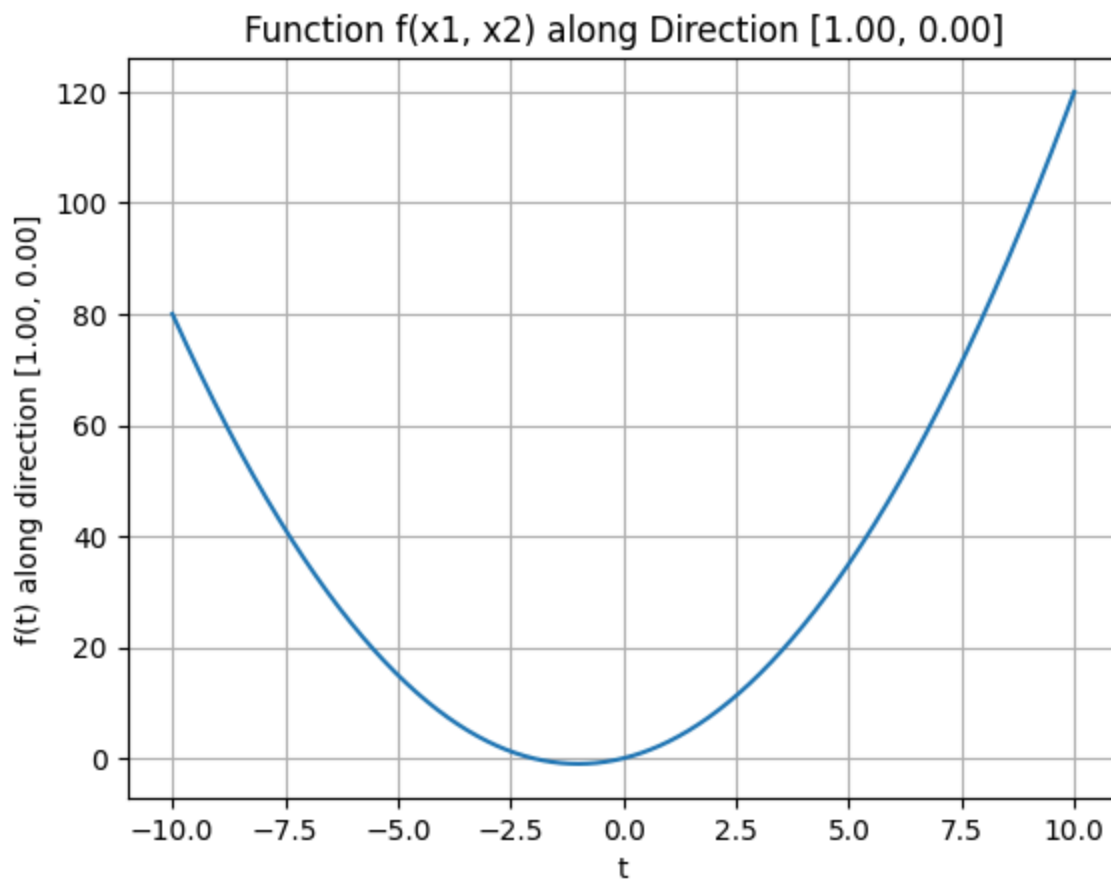


Function $f(x_1, x_2)$ along Direction $[-0.15, 0.29]$



Function $f(x_1, x_2)$ along Direction $[-0.12, 0.78]$





As can be seen above, for all the random directions, the function $f(x_1, x_2)$ restricted to the lines going through the point $(0, 0)$ and in those directions is always convex.

(b)

The gradient of the function is:

$$\nabla f(x_1, x_2) = \begin{bmatrix} \frac{\partial f(x_1, x_2)}{\partial x_1} \\ \frac{\partial f(x_1, x_2)}{\partial x_2} \end{bmatrix} = \begin{bmatrix} 2x_1 + 3x_2 + 2 \\ 3x_1 + 18x_2 - 5 \end{bmatrix}$$

Then, the gradient descent is implemented below. The step sizes (found experimentally are):

- Too small: $\gamma = 0.005$
- Too large: $\gamma = 0.11$
- Just right: $\gamma = 0.05$

In [172...

```
# Calculate the gradient of f (use the formula derived above)
```

```
def grad_f(x):
    x1, x2 = x
    df_dx1 = 2 * x1 + 3 * x2 + 2
    df_dx2 = 3 * x1 + 18 * x2 - 5
    return np.array([df_dx1, df_dx2])

# Run gradient descent
def gradient_descent(x0, gamma, n_iter):
    x = np.array(x0, dtype=float)
    x_history = [x.copy()]
    for _ in range(n_iter):
        # Calculate gradients and update x
        grad = grad_f(x)
```

```

    x = x - gamma * grad
    x_history.append(x.copy())
    # Return the evolution of x over time
    return np.array(x_history)

# Parameters for GD
x0 = np.random.uniform(-10, 10, (2)) # Start at random
n_iter = 500 # Number of iterations
gammas = [0.005, 0.05, 0.11] # Step sizes
labels = [r"$\gamma=0.005", r"$\gamma=0.05", r"$\gamma=0.11"] # Labels for the plot

# Plot the 3 GD cases
# For each gamma, show how x1(k) and x2(k) evolve and the contour with the trajectory
fig, axes = plt.subplots(3, 2, figsize=(12, 18))
for i, gamma in enumerate(gammas):
    x_history = gradient_descent(x0, gamma, n_iter)
    print(f"Final x for step size {gammas[i]}: {x_history[-1][0]:.3f}, {x_history[-1][1]:.3f}")
    if i == 1:
        x_b_star = x_history[-1]
    # Evolution of x1 and x2 over k
    k = np.arange(n_iter + 1)
    axes[i, 0].plot(k, x_history[:, 0], "b-", label="x1")
    axes[i, 0].plot(k, x_history[:, 1], "r-", label="x2")
    axes[i, 0].set_xlabel("Iteration k")
    axes[i, 0].set_ylabel("x1, x2")
    axes[i, 0].legend()
    axes[i, 0].grid(True)
    axes[i, 0].set_title(f"Evolution x1(k), x2(k) for step size {labels[i]}")

    # Contour with trajectory
    max_lim = np.max(np.abs(x_history)) + 1
    x1 = np.linspace(-max_lim, max_lim, 1000)
    x2 = np.linspace(-max_lim, max_lim, 1000)
    X1, X2 = np.meshgrid(x1, x2)
    Z = f(X1, X2)
    axes[i, 1].contour(X1, X2, Z, levels=25, cmap="viridis")
    axes[i, 1].plot(x_history[:, 0], x_history[:, 1], "ro-", markersize=3, label="Trajectory")
    axes[i, 1].plot(x0[0], x0[1], "bo", label="Start")
    axes[i, 1].set_xlabel("x1")
    axes[i, 1].set_ylabel("x2")
    axes[i, 1].legend()
    axes[i, 1].set_title(f"Trajectory of x over time for step size {labels[i]}")

plt.tight_layout()
plt.show()
plt.close()

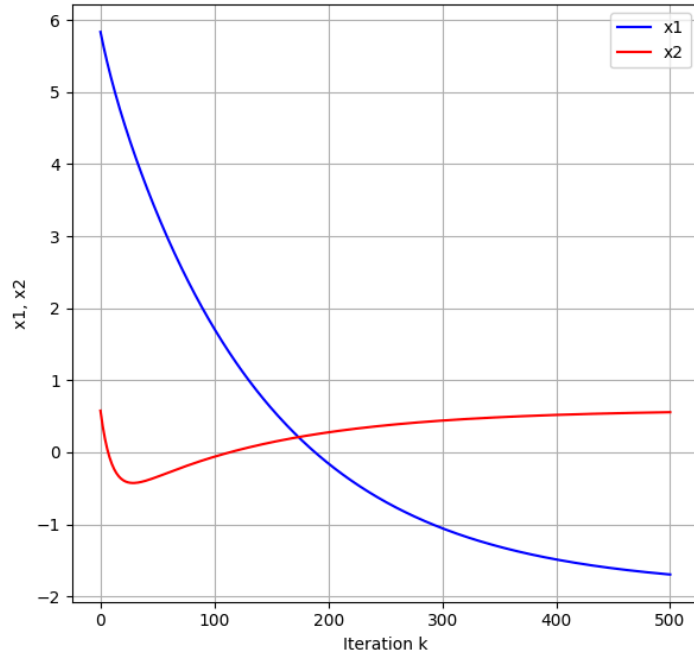
```

Final x for step size 0.005: -1.695, 0.557

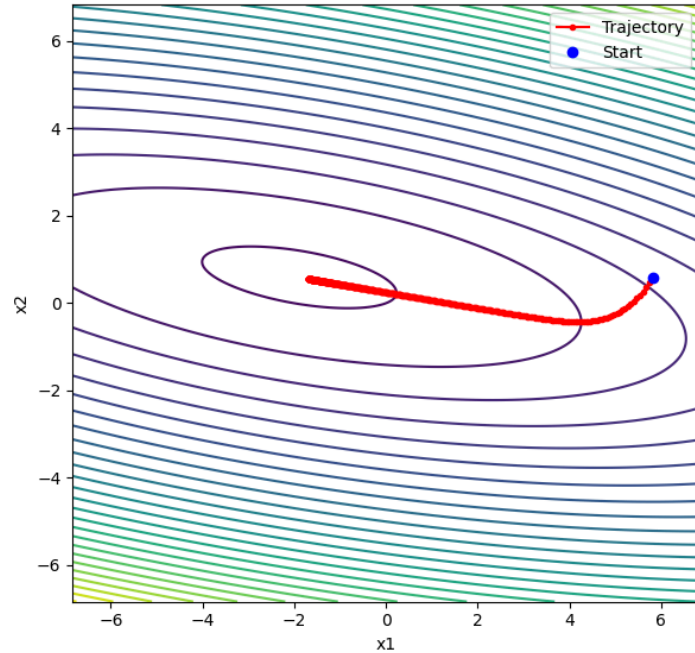
Final x for step size 0.05: -1.889, 0.593

Final x for step size 0.11: 74042924.565, 408322151.449

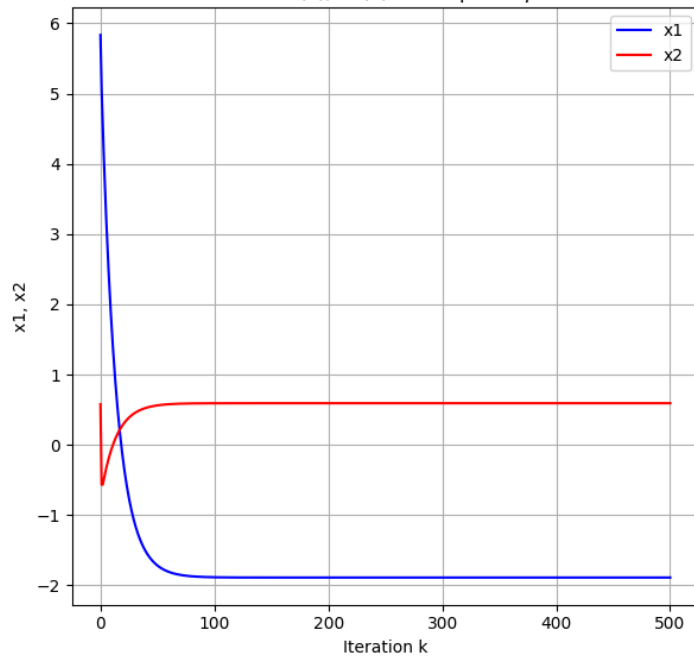
Evolution $x_1(k)$, $x_2(k)$ for step size $\gamma=0.005$



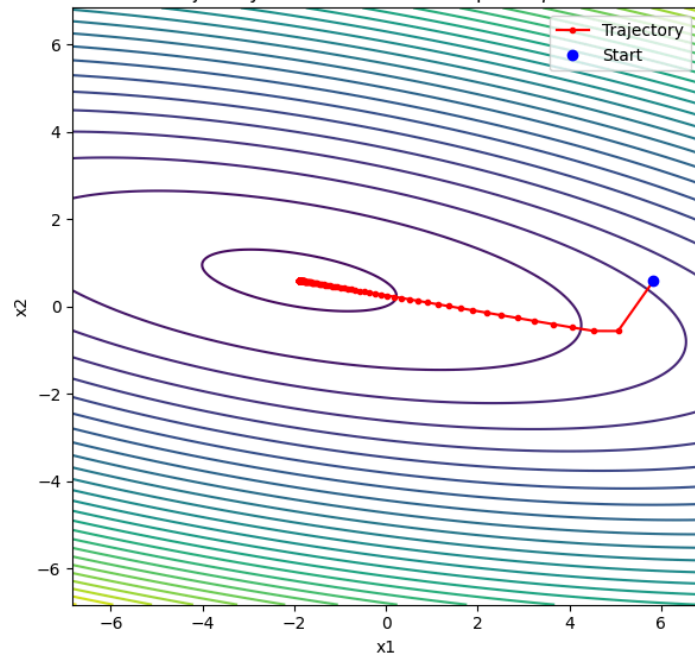
Trajectory of x over time for step size $\gamma=0.005$



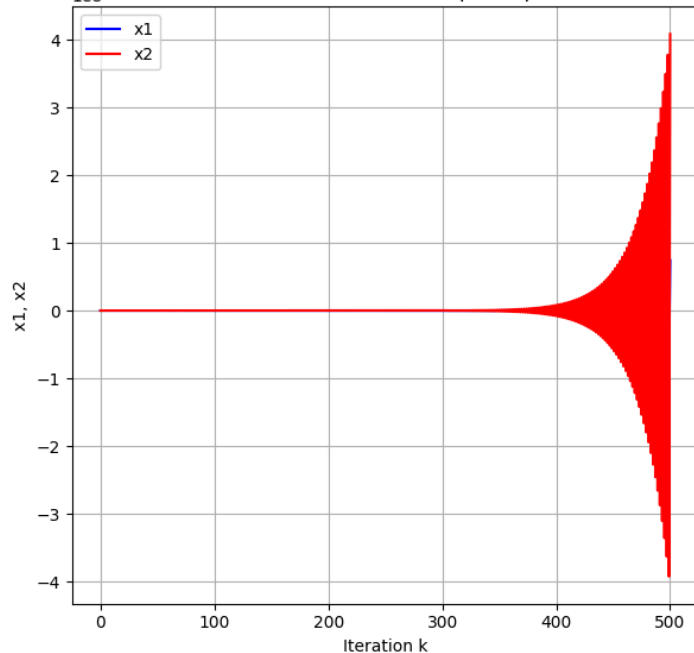
Evolution $x_1(k)$, $x_2(k)$ for step size $\gamma=0.05$



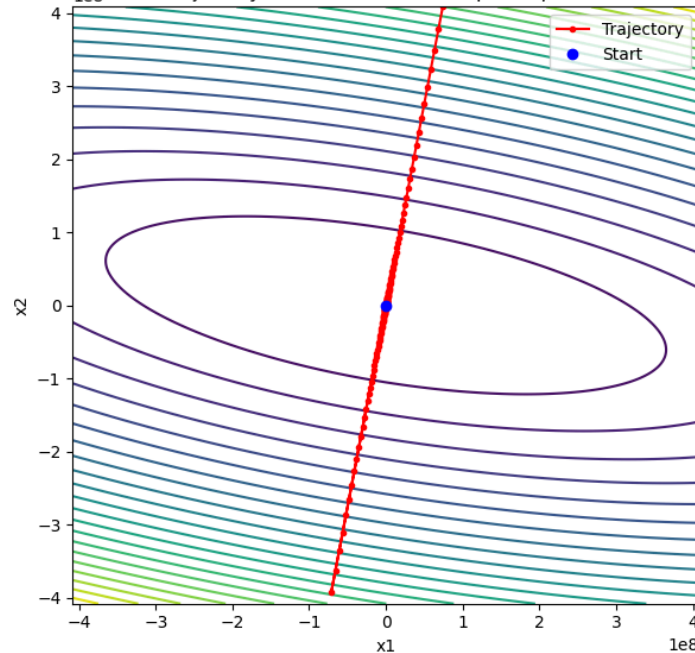
Trajectory of x over time for step size $\gamma=0.05$



Evolution $x_1(k)$, $x_2(k)$ for step size $\gamma=0.11$



Trajectory of x over time for step size $\gamma=0.11$



(c)

Now the problem is:

$$\begin{aligned} \min_{x_1, x_2} \quad & x_1^2 + 3x_1x_2 + 9x_2^2 + 2x_1 - 5x_2 \\ \text{subject to} \quad & 2x_1 + x_2 \geq 3 \\ & x_1 + 2x_2 \geq 3 \\ & x_1 \geq 0 \\ & x_2 \geq 0 \end{aligned}$$

Just as suggested, the following is the gradient projection algorithm, which is the same as above, but at every step, the point $x = (x_1, x_2)$ is projected onto the feasible set.

Now, to calculate the projections, we know the feasible set is in the first quadrant and above the lines

- $2x_1 + x_2 = 3$
- $x_1 + 2x_2 = 3$

Next, the lines normal to these lines are (at the boundary points):

- $-0.5x_1 + x_2 = 3$ perpendicular to $2x_1 + x_2 = 3$ at $(0, 3)$
- $x_1 - 0.5x_2 = 3$ perpendicular to $x_1 + 2x_2 = 3$ at $(3, 0)$

We need these, because the projection rules can be graphically illustrated as follows:



From the figure above, there are 7 distinct cases for projection when the point is outside the feasible set (of course, if the point is inside the feasible set, the projection is the point itself). The projection rules are as follows:

- Case 1: $x_1 < 0$ and $x_2 \geq 3 \Rightarrow$ project to $(0, x_2)$
- Case 2: $x_2 < 3$ and $-0.5x_1 + x_2 \geq 3 \Rightarrow$ project to $(0, 3)$
- Case 3: $-0.5x_1 + x_2 < 3$ and $-0.5x_1 + x_2 \geq 0.5 \Rightarrow$ project to $(\frac{x_1 - 2x_2 + 6}{5}, \frac{-2x_1 + 4x_2 + 3}{5})$ (onto the line $2x_1 + x_2 = 3$, perpendicular projection)
- Case 4: $-0.5x_1 + x_2 < 0.5$ and $x_1 - 0.5x_2 \leq 0.5 \Rightarrow$ project to $(1, 1)$
- Case 5: $x_1 - 0.5x_2 > 0.5$ and $x_1 - 0.5x_2 \leq 3 \Rightarrow$ project to $(\frac{4x_1 - 2x_2 + 3}{5}, \frac{-2x_1 + x_2 + 6}{5})$ (onto the line $x_1 + 2x_2 = 3$, perpendicular projection)
- Case 6: $x_1 - 0.5x_2 > 3$ and $x_1 < 3 \Rightarrow$ project to $(3, 0)$
- Case 7: $x_2 < 0$ and $x_1 \geq 3 \Rightarrow$ project to $(x_1, 0)$

With that, I added the projection function that implements these rules, and the rest remains the same. The code and results are below.

The step sizes used (found experimentally) are:

- Too small: $\gamma = 0.001$
- Too large: $\gamma = 10.0$
- Just right: $\gamma = 0.05$


```

# Project onto the feasible set
def project(x):
    x1, x2 = x
    if x1 >= 0 and x2 >= 0 and 2 * x1 + x2 >= 3 and x1 + 2 * x2 >= 3:
        return x
    # The next two are the most likely to be used so put them first
    elif x1 < 0 and x2 >= 3:
        return np.array([0, x2])
    elif x2 < 0 and x1 >= 3:
        return np.array([x1, 0])
    # Then the smaller regions in order of size (based on probability for optimality)
    elif -0.5 * x1 + x2 <= 0.5 and x1 - 0.5 * x2 <= 0.5:
        return np.array([1, 1])
    elif x2 <= 3 and -0.5 * x1 + x2 >= 3:
        return np.array([0, 3])
    elif x1 <= 3 and x1 - 0.5 * x2 >= 3:
        return np.array([3, 0])
    elif -0.5 * x1 + x2 <= 3 and -0.5 * x1 + x2 >= 0.5:
        return np.array([(x1 - 2 * x2 + 6) / 5, (-2 * x1 + 4 * x2 + 3) / 5])
    elif x1 - 0.5 * x2 >= 0.5 and x1 - 0.5 * x2 <= 3:
        return np.array([(4 * x1 - 2 * x2 + 3) / 5, (-2 * x1 + x2 + 6) / 5])

# Run projected gradient descent
def projected_gradient_descent(x0, gamma, n_iter):
    x = np.array(x0, dtype=float)
    x_history = [x.copy()]
    for _ in range(n_iter):
        # Calculate gradients and update x
        grad = grad_f(x)
        x = x - gamma * grad
        x = project(x) # Project onto the feasible set
        x_history.append(x.copy())
    # Return the evolution of x over time
    return np.array(x_history)

# Parameters for projected-GD
x0 = project(np.random.uniform(-10, 10, (2))) # Start at random
n_iter = 500 # Number of iterations
gammas = [0.001, 0.05, 10] # Step sizes
labels = [r"$\gamma=0.001", r"$\gamma=0.05", r"$\gamma=10"] # Labels for the plot

# Plot the 3 projected-GD cases
# For each gamma, show how x1(k) and x2(k) evolve and the contour with the trajectory
fig, axes = plt.subplots(3, 2, figsize=(12, 18))
for i, gamma in enumerate(gammas):
    x_history = projected_gradient_descent(x0, gamma, n_iter)
    print(f"Final x for step size {gammas[i]}: {x_history[-1][0]:.3f}, {x_history[-1][1]:.3f}")
    if i == 1:
        x_c_star = x_history[-1]
    # Evolution of x1 and x2 over k
    k = np.arange(n_iter + 1)
    axes[i, 0].plot(k, x_history[:, 0], "b-", label="x1")
    axes[i, 0].plot(k, x_history[:, 1], "r-", label="x2")
    axes[i, 0].set_xlabel("Iteration k")
    axes[i, 0].set_ylabel("x1, x2")
    axes[i, 0].legend()
    axes[i, 0].grid(True)
    axes[i, 0].set_title(f"Evolution x1(k), x2(k) for step size {labels[i]}")

# Contour with trajectory
max_lim = np.max(np.abs(x_history)) + 1

```

```

x1 = np.linspace(-max_lim, max_lim, 1000)
x2 = np.linspace(-max_lim, max_lim, 1000)
X1, X2 = np.meshgrid(x1, x2)
Z = f(X1, X2)
axes[i, 1].contour(X1, X2, Z, levels=25, cmap="viridis")
axes[i, 1].plot(x_history[:, 0], x_history[:, 1], "ro-", markersize=3, label="Trajectory")
axes[i, 1].plot(x0[0], x0[1], "bo", label="Start")
axes[i, 1].set_xlabel("x1")
axes[i, 1].set_ylabel("x2")
axes[i, 1].legend()
axes[i, 1].set_title(f"Trajectory of x over time for step size {labels[i]}")

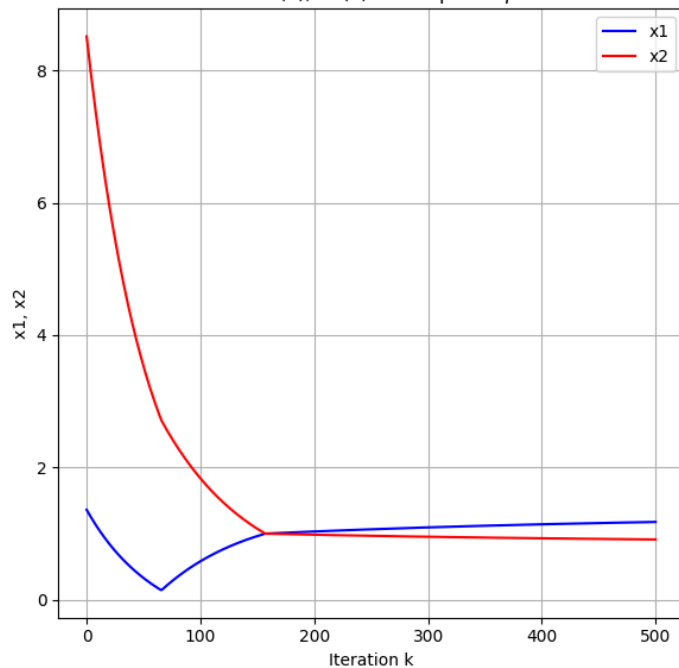
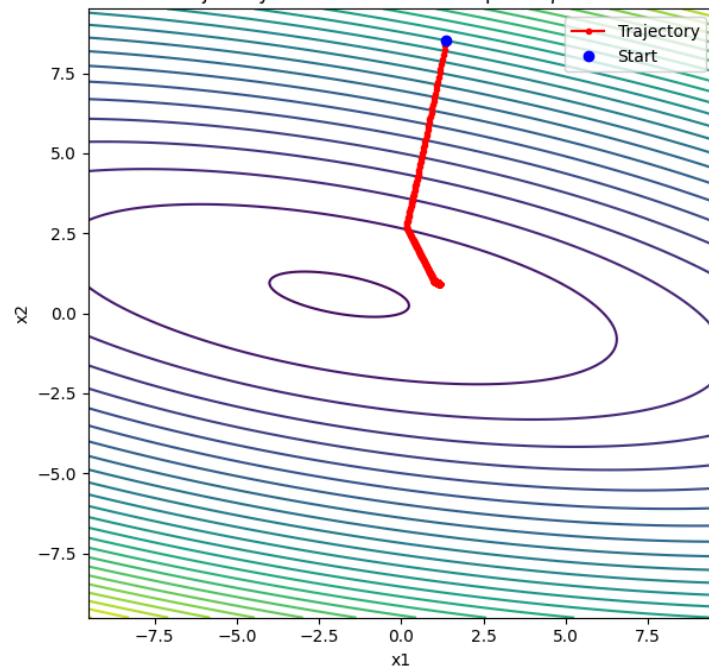
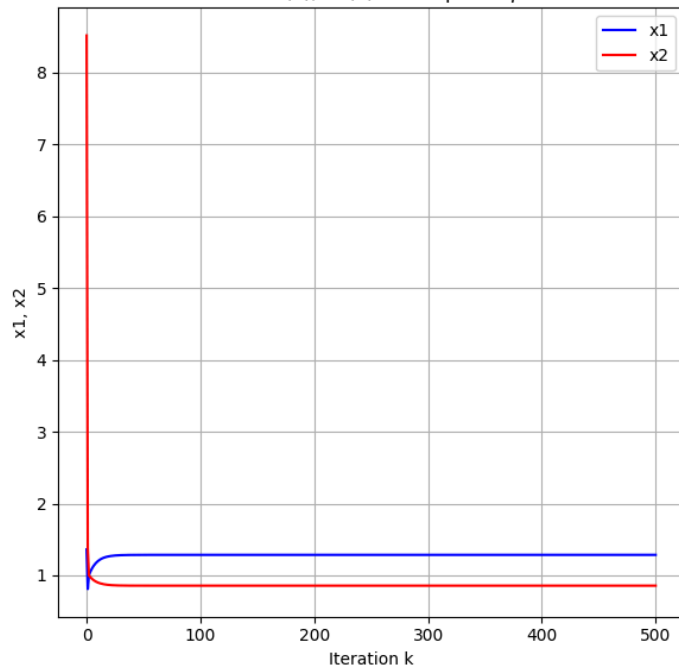
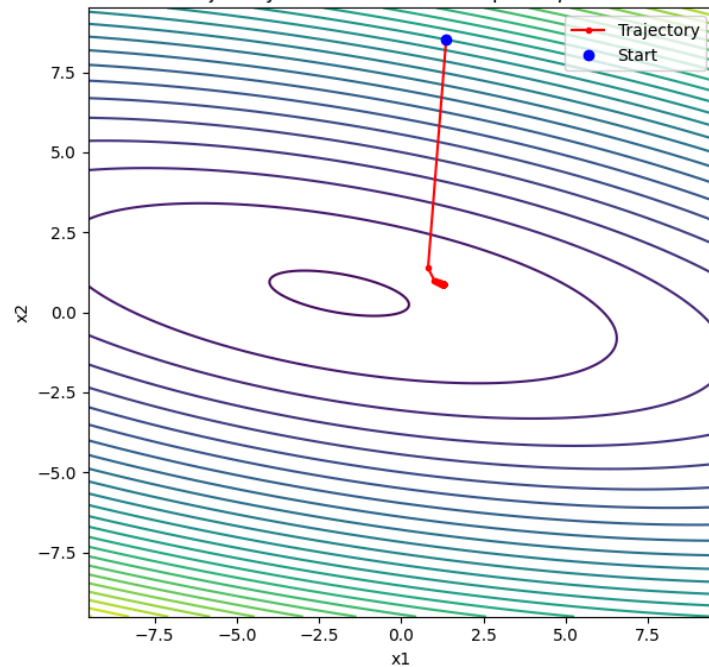
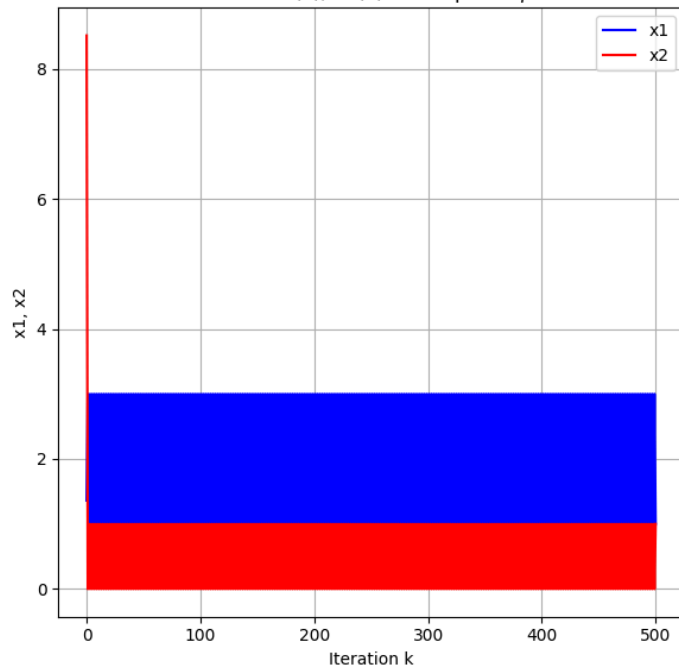
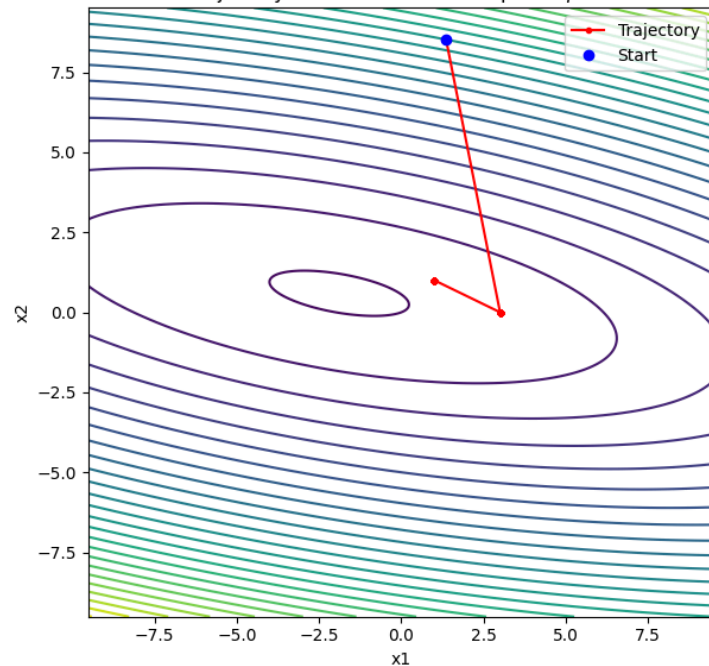
plt.tight_layout()
plt.show()
plt.close()

```

Final x for step size 0.001: 1.177, 0.912

Final x for step size 0.05: 1.286, 0.857

Final x for step size 10: 1.000, 1.000

Evolution $x_1(k)$, $x_2(k)$ for step size $\gamma=0.001$ Trajectory of x over time for step size $\gamma=0.001$ Evolution $x_1(k)$, $x_2(k)$ for step size $\gamma=0.05$ Trajectory of x over time for step size $\gamma=0.05$ Evolution $x_1(k)$, $x_2(k)$ for step size $\gamma=10$ Trajectory of x over time for step size $\gamma=10$ 

(d)

The lagrangian is:

$$\mathcal{L}(x_1, x_2, \lambda_1, \lambda_2) = x_1^2 + 3x_1x_2 + 9x_2^2 + 2x_1 - 5x_2 + \lambda_1(3 - 2x_1 - x_2) + \lambda_2(3 - x_1 - 2x_2)$$

(the $x_1 \geq 0$ and $x_2 \geq 0$ are not included in the lagrangian, and are enforced separately through projection).

The lagrangian can be rewritten as:

$$\mathcal{L}(x_1, x_2, \lambda_1, \lambda_2) = x_1^2 + 3x_1x_2 + 9x_2^2 + (2 - 2\lambda_1 - \lambda_2)x_1 + (-5 - \lambda_1 - 2\lambda_2)x_2 + 3\lambda_1 + 3\lambda_2$$

This is a convex quadratic function in x_1 and x_2 , so we can find the x_1 and x_2 that minimize the lagrangian by setting the gradient to zero. The gradient is:

$$\nabla \mathcal{L}(x_1, x_2, \lambda_1, \lambda_2) = \begin{bmatrix} 2x_1 + 3x_2 + 2 - 2\lambda_1 - \lambda_2 \\ 3x_1 + 18x_2 - 5 - \lambda_1 - 2\lambda_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Solving this, we get:

$$\begin{aligned} x_1^* &= \frac{11\lambda_1 + 4\lambda_2 - 17}{9} \\ x_2^* &= \frac{-4\lambda_1 + \lambda_2 + 16}{27} \end{aligned}$$

Also, we have the non-negativity constraints, so the actual optimal values of x_1 and x_2 are the projections of the above onto the first quadrant:

- If $x_1^* < 0$, $x_1^* = 0$, minimize $\mathcal{L}(0, x_2^*, \lambda_1, \lambda_2)$ over $x_2^* \Rightarrow x_2^* = \frac{5+\lambda_1+2\lambda_2}{18}$
- If $x_2^* < 0$, $x_2^* = 0$, minimize $\mathcal{L}(x_1^*, 0, \lambda_1, \lambda_2)$ over $x_1^* \Rightarrow x_1^* = \frac{-2+2\lambda_1+\lambda_2}{2}$

Given that, the dual problem is

$$\begin{aligned} \max_{\lambda_1, \lambda_2} g(\lambda_1, \lambda_2) &= \max_{\lambda_1, \lambda_2} \mathcal{L}(x_1^*, x_2^*, \lambda_1, \lambda_2) \\ \text{subject to } \lambda_1 &\geq 0 \\ \lambda_2 &\geq 0 \end{aligned}$$

Then, the projected gradient ascent maximizes the $g(\lambda_1, \lambda_2)$ by projecting back to $\lambda_1 \geq 0$ and $\lambda_2 \geq 0$.

The gradient of $g(\lambda_1, \lambda_2)$ is:

$$\nabla g(\lambda_1, \lambda_2) = \begin{bmatrix} 3 - 2x_1^* - x_2^* \\ 3 - x_1^* - 2x_2^* \end{bmatrix}$$

Given that, I use the above formula in the projected gradient ascent (along with projecting the x back to the feasible set). The implementation is below.

The step sizes used (found experimentally) are:

- Too small: $\gamma = 0.01$
- Too large: $\gamma = 3.0$
- Just right: $\gamma = 0.3$

```

def grad_lambda(x):
    x1_star, x2_star = x
    dg_dlambda1 = 3 - 2 * x1_star - x2_star
    dg_dlambda2 = 3 - x1_star - 2 * x2_star
    return np.array([dg_dlambda1, dg_dlambda2])

def calc_x_star(lambdas):
    lambda1, lambda2 = lambdas
    x1_star = (11 * lambda1 + 4 * lambda2 - 17) / 9
    x2_star = (-4 * lambda1 + lambda2 + 16) / 27
    if x1_star < 0:
        x1_star = 0
        x2_star = max(0, (5 * lambda1 + lambda2 + 2) / 18)
    if x2_star < 0:
        x1_star = max(0, (-2 * lambda1 + 2 * lambda2 + 1) / 2)
        x2_star = 0
    if x1_star < 0 and x2_star < 0:
        x1_star = 0
        x2_star = 0

    return np.array([x1_star, x2_star])

def project_lambda(lambdas):
    lambda1, lambda2 = lambdas
    if lambda1 < 0:
        lambda1 = 0
    if lambda2 < 0:
        lambda2 = 0
    return np.array([lambda1, lambda2])

# Run projected gradient descent
def projected_dual_gradient_descent(lambdas0, gamma, n_iter):
    lambdas = np.array(lambdas0, dtype=float)
    x_history = []
    lambdas_history = [lambdas.copy()]

    for _ in range(n_iter):
        # Calculate gradients and update Lambdas
        x = calc_x_star(lambdas)
        x_history.append(x.copy())
        grad = grad_lambda(x)
        lambdas = lambdas + gamma * grad # Max -> ascent
        lambdas = project_lambda(lambdas) # Project onto the feasible set
        lambdas_history.append(lambdas.copy())
    x_history.append(calc_x_star(lambdas)) # Final x_star
    # Return the evolution of Lambdas over time
    return np.array(lambdas_history), np.array(x_history)

# Parameters for projected-GD
lambdas0 = np.random.uniform(0, 10, (2)) # Start at random
n_iter = 500
gammas = [0.01, 0.3, 3.0] # Step sizes
labels = [r"$\gamma=0.01", r"$\gamma=0.3", r"$\gamma=3"] # Labels for the plot

# Plot the 3 projected-GD cases
# For each gamma, show how x1(k) and x2(k) evolve and the contour with the trajectory
fig, axes = plt.subplots(3, 2, figsize=(12, 18))
for i, gamma in enumerate(gammas):
    lambdas_history, x_history = projected_dual_gradient_descent(lambdas0, gamma, n_iter)
    print(f"Final x for step size {gammas[i]}: {x_history[-1][0]:.3f}, {x_history[-1][1]:.3f}")
    print(f"Final lambdas for step size {gammas[i]}: {lambdas_history[-1][0]:.3f}, {lambdas_hist

```

```

print()
if i == 1:
    x_d_star = x_history[-1]
    lambdas_star = lambdas_history[-1]
# Evolution of x1 and x2 over k
k = np.arange(n_iter + 1)
axes[i, 0].plot(k, x_history[:, 0], "b-", label="x1")
axes[i, 0].plot(k, x_history[:, 1], "r-", label="x2")
axes[i, 0].plot(k, lambdas_history[:, 0], "-", label=r"$\lambda_1$")
axes[i, 0].plot(k, lambdas_history[:, 1], "-", label=r"$\lambda_2$")
axes[i, 0].set_xlabel("Iteration k")
axes[i, 0].set_ylabel("x1, x2, lambdas")
axes[i, 0].legend()
axes[i, 0].grid(True)
axes[i, 0].set_title(f"Evolution x1(k), x2(k) for step size {labels[i]}")

# Contour with trajectory
max_lim = np.max(np.abs(x_history)) + 1
x1 = np.linspace(-max_lim, max_lim, 1000)
x2 = np.linspace(-max_lim, max_lim, 1000)
X1, X2 = np.meshgrid(x1, x2)
Z = f(X1, X2)
axes[i, 1].contour(X1, X2, Z, levels=25, cmap="viridis")
axes[i, 1].plot(
    x_history[:, 0], x_history[:, 1], "ro-", markersize=3, label="Trajectory"
)
axes[i, 1].plot(x_history[0, 0], x_history[0, 1], "bo", label="Start")
axes[i, 1].set_xlabel("x1")
axes[i, 1].set_ylabel("x2")
axes[i, 1].legend()
axes[i, 1].set_title(f"Trajectory of x over time for step size {labels[i]}")

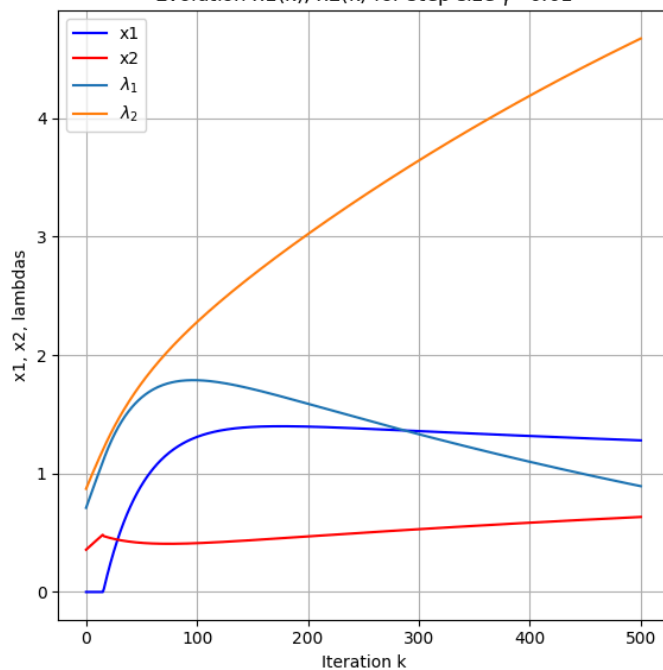
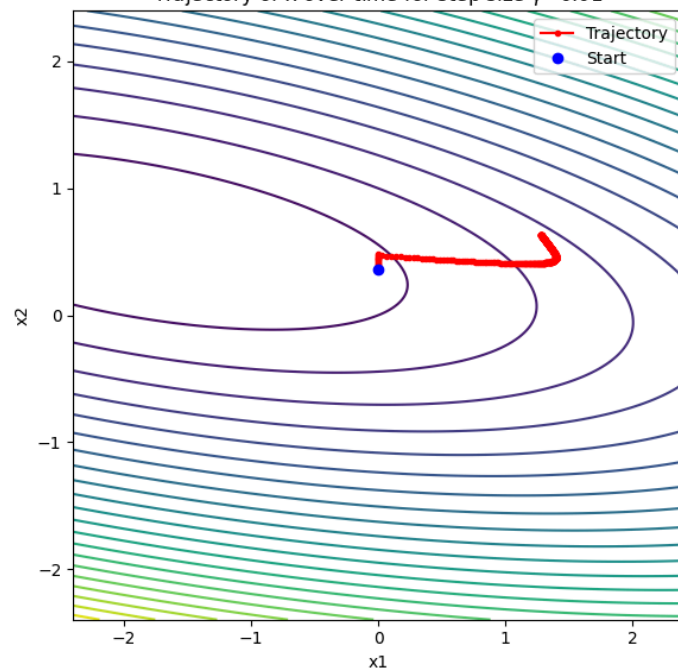
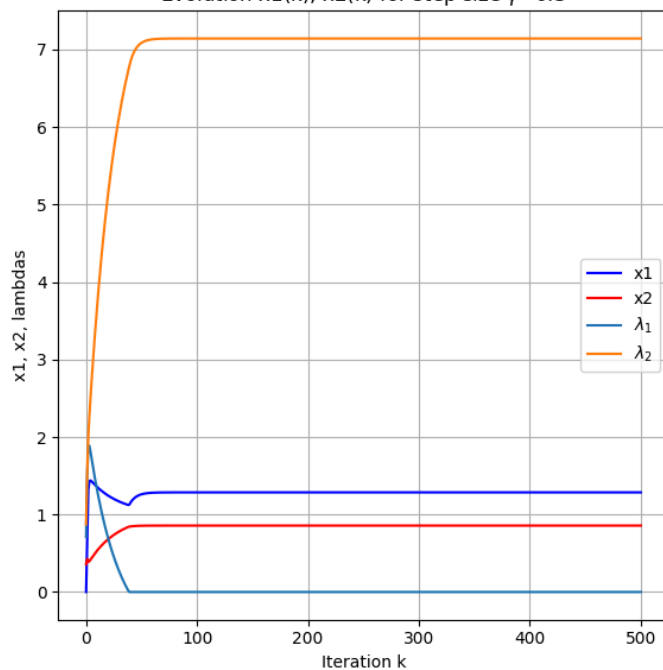
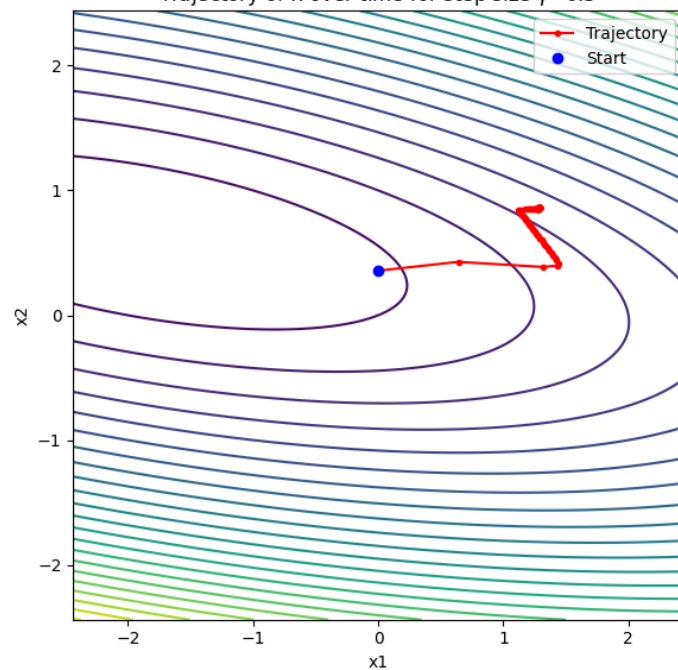
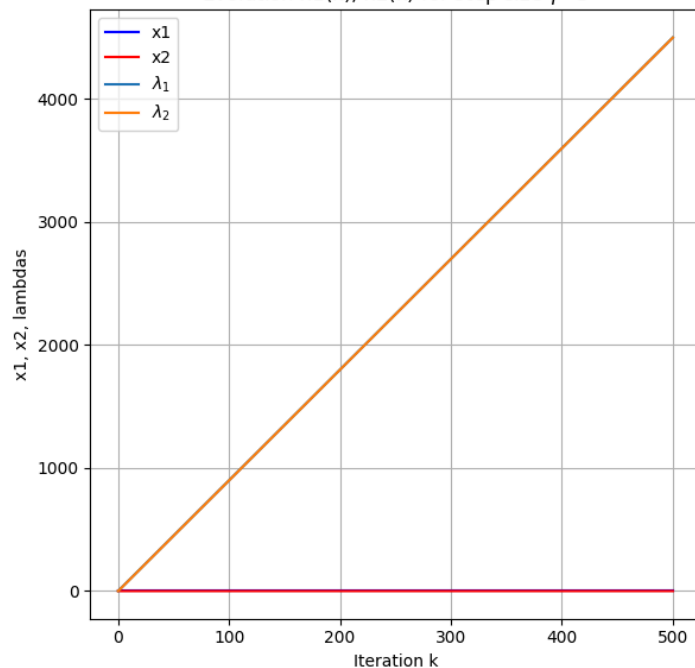
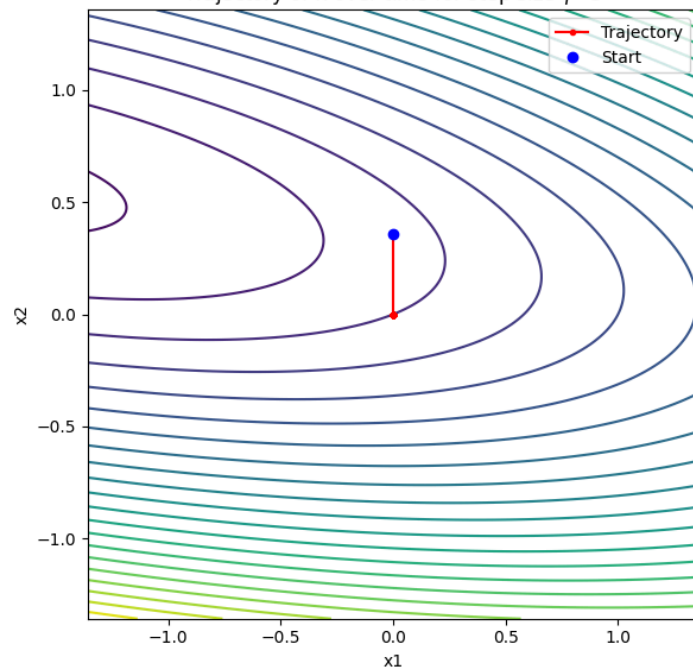
plt.tight_layout()
plt.show()
plt.close()

```

Final x for step size 0.01: 1.280, 0.633
 Final lambdas for step size 0.01: 0.893, 4.674

Final x for step size 0.3: 1.286, 0.857
 Final lambdas for step size 0.3: 0.000, 7.143

Final x for step size 3.0: 0.000, 0.000
 Final lambdas for step size 3.0: 4499.640, 4498.730

Evolution $x_1(k)$, $x_2(k)$ for step size $\gamma=0.01$ Trajectory of x over time for step size $\gamma=0.01$ Evolution $x_1(k)$, $x_2(k)$ for step size $\gamma=0.3$ Trajectory of x over time for step size $\gamma=0.3$ Evolution $x_1(k)$, $x_2(k)$ for step size $\gamma=3$ Trajectory of x over time for step size $\gamma=3$ 

(e)

First of all, the Hessian $\nabla^2 f(x_1, x_2)$ is:

$$\nabla^2 f(x_1, x_2) = \begin{bmatrix} 2 & 3 \\ 3 & 18 \end{bmatrix}$$

. Since the matrix is symmetric, and its eigenvalues are $10 - \sqrt{73}$ and $10 + \sqrt{73}$, both of which are positive, the Hessian is positive definite. So the function f is convex.

Then, to check if we have an optimal solution, for cases (b) and (c) I use the first order condition, checking if $\nabla f^T d \geq 0$ for all directions d in the feasible set (since I can't check all directions (uncountable), I check a number of random directions). For (d) I just check if the KKT conditions hold (then the optimality gap is 0, and from (b) and (d) we can check that the primal solution is optimal).

In [175...

```
def check_first_order_cond(x, num_pts=10000, conditions=None):
    grad_f_at_x = grad_f(x)
    # Only choose points in the square [0, 10] x [0, 10]
    for _ in range(num_pts):
        d = np.random.uniform(0, 10, (2))
        if conditions is not None:
            cond_d = [cond(d[0], d[1]) for cond in conditions]
        else:
            cond_d = [True, True]
        # If didnt satisfy, keep choosing until satisfied
        while not np.all(np.all(cond_d)):
            d = np.random.uniform(0, 10, (2))
            cond_d = [cond(d[0], d[1]) for cond in conditions]
        # Check the first order condition
        if np.dot(grad_f_at_x, d) < 0 - 1e-8:
            return False
    return True

def KKT_conditions(x, lambdas, f, f_is, lagrangian, num_pts=10000):
    x1, x2 = x
    if_all_true = True
    for f_i in f_is:
        if f_i(x1, x2) > 0 + 1e-8:
            if_all_true = False
            return if_all_true
    for lambda_ in lambdas:
        if lambda_ < 0:
            if_all_true = False
            return if_all_true
    for lambda_, f_i in zip(lambdas, f_is[:len(lambdas)]):
        if lambda_ * f_i(x1, x2) > 0 + 1e-8 or lambda_ * f_i(x1, x2) < 0 - 1e-8:
            if_all_true = False
            return if_all_true
    lagrangian_star = lagrangian(x1, x2, lambdas)
    for _ in range(num_pts):
        x_test = np.random.uniform(0, 10, (2))
        if lagrangian(x_test[0], x_test[1], lambdas) < lagrangian_star:
            if_all_true = False
            return if_all_true
    return if_all_true

# Check (b)
if_b_optimal = check_first_order_cond(x_b_star, num_pts=10000)
```



```

print(f"Is x_b_star = [{x_b_star[0]:4f}, {x_b_star[1]:4f}] optimal? {if_b_optimal}")

# Check (c)
conditions = [
    lambda x, y: x >= 0 and y >= 0,
    lambda x, y: 2 * x + y >= 3,
    lambda x, y: x + 2 * y >= 3,
]
if_c_optimal = check_first_order_cond(x_c_star, num_pts=10000, conditions=conditions)
print(f"Is x_c_star = [{x_c_star[0]:4f}, {x_c_star[1]:4f}] optimal? {if_c_optimal}")

# Check (d)
lagrangian = lambda x1, x2, lambdas: f(x1, x2) + lambdas[0] * (3 - 2 * x1 - x2) + lambdas[1] * (
f_is = [
    lambda x1, x2: 3 - 2 * x1 - x2,
    lambda x1, x2: 3 - x1 - 2 * x2,
    lambda x1, x2: -x1,
    lambda x1, x2: -x2
]
if_d_optimal = KKT_conditions(x_d_star, lambdas_star, f, f_is, lagrangian, num_pts=10000)
print(f"Is x_d_star = [{x_d_star[0]:4f}, {x_d_star[1]:4f}] optimal? {if_d_optimal}")

```

```

Is x_b_star = [-1.888889, 0.592593] optimal? True
Is x_c_star = [1.285714, 0.857143] optimal? True
Is x_d_star = [1.285714, 0.857143] optimal? True

```

So as can be seen above, the (probabilistic) checks confirm that most likely, the solutions found are optimal.

Problem 2

(a)

First, the goal is to maximize the total utility. Each flow i has a utility:

$$U_i(x_i) = w_i \ln(x_i)$$

where x_i is the rate of the flow i and w_i is its weight.

The weights are: $w = [w_i] = [1, 1, 1, 1, 2, 2, 2]$.

The flows are subject to capacity constraints - there are 12 link with capacities

$c = [c_j] = [1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2]$. The flow paths are:

- Flow 1: $3 \rightarrow 9$
- Flow 2: $4 \rightarrow 9$
- Flow 3: $3 \rightarrow 10$
- Flow 4: $4 \rightarrow 10$
- Flow 5: $5 \rightarrow 8$
- Flow 6: $5 \rightarrow 9$
- Flow 7: $1 \rightarrow 6 \rightarrow 11$

As a result, we can define the flow matrix $A_{12 \times 7}$ where $A_{l,i}$ is 1 if the flow i passes through link l and 0 otherwise. So the following are 1:

- $A_{3,1} = 1, A_{9,1} = 1$
- $A_{4,2} = 1, A_{9,2} = 1$
- $A_{3,3} = 1, A_{10,3} = 1$

- $A_{4,4} = 1, A_{10,4} = 1$
- $A_{5,5} = 1, A_{8,5} = 1$
- $A_{5,6} = 1, A_{9,6} = 1$
- $A_{1,7} = 1, A_{6,7} = 1, A_{11,7} = 1$

For each link l , the constraint is then:

$$\sum_{i=1}^7 A_{l,i} x_i \leq c_l, \text{ for } l = 1, \dots, 12$$

which can be expressed in the matrix notation as:

$$Ax \leq c$$

Then, the constrained optimization problem is:

$$\begin{aligned} \max_x \quad & \sum_{i=1}^7 w_i \ln(x_i) \\ \text{subject to} \quad & Ax \leq c \\ & x_i \geq 0, i = 1, \dots, 7 \end{aligned}$$

which is equivalent to the following minimization problem:

$$\begin{aligned} \min_x \quad & \sum_{i=1}^7 -w_i \ln(x_i) \\ \text{subject to} \quad & Ax \leq c \\ & x_i \geq 0, i = 1, \dots, 7 \end{aligned}$$

where $x = [x_i]^T$ and $c = [c_j]^T$ are the flow and resource capacity vectors, respectively.

Next, for the dual problem, let $\lambda = [\lambda_j]$ be the Lagrange multipliers. The Lagrangian is:

$$\begin{aligned} \mathcal{L}(x, \lambda) &= \sum_{i=1}^7 -w_i \ln(x_i) + \sum_{l=1}^{12} \lambda_l \left(\sum_{i=1}^7 A_{l,i} x_i - c_l \right) \\ &= \sum_{i=1}^7 -w_i \ln(x_i) + \sum_{l=1}^{12} \lambda_l \sum_{i=1}^7 A_{l,i} x_i - \sum_{l=1}^{12} \lambda_l c_l \end{aligned}$$

(the non-negativity of x_i is enforced implicitly (not in the Lagrangian)).

Next, define:

$$g(\lambda) = \min_x \mathcal{L}(x, \lambda) = \min_x \sum_{i=1}^7 -w_i \ln(x_i) + \sum_{l=1}^{12} \lambda_l \sum_{i=1}^7 A_{l,i} x_i - \sum_{l=1}^{12} \lambda_l c_l$$

Note that this function is convex in x ($-\ln(x)$ is convex, and the sum is linear in x), so to find the x that minimizes this function, I can take the gradient and set it to 0. Also this is separable in x_i , so I can take the partial derivative w.r.t. x_i and set it to 0 for each i (and the answer will be the same). Then:

$$\frac{\partial \mathcal{L}(x, \lambda)}{\partial x_i} = -\frac{w_i}{x_i} + \sum_{l=1}^{12} \lambda_l A_{l,i} = 0$$

$$\iff x_i^* = \frac{w_i}{\sum_{l=1}^{12} \lambda_l A_{l,i}}$$

Substituting x_i^* back into the lagrangian:

$$\begin{aligned} g(\lambda) &= \mathcal{L}(x^*, \lambda) = \sum_{i=1}^7 -w_i \ln\left(\frac{w_i}{\sum_{l=1}^{12} \lambda_l A_{l,i}}\right) + \sum_{l=1}^{12} \lambda_l \sum_{i=1}^7 \frac{A_{l,i} w_i}{\sum_{k=1}^{12} \lambda_k A_{k,i}} - \sum_{l=1}^{12} \lambda_l c_l \\ &= -\sum_{i=1}^7 w_i \ln(w_i) + \sum_{i=1}^7 w_i \ln\left(\sum_{l=1}^{12} \lambda_l A_{l,i}\right) + \sum_{l=1}^{12} \lambda_l \sum_{i=1}^7 \frac{A_{l,i} w_i}{\sum_{k=1}^{12} \lambda_k A_{k,i}} - \sum_{l=1}^{12} \lambda_l c_l \\ &= \text{const} + \sum_{i=1}^7 \left[w_i \ln\left(\sum_{l=1}^{12} \lambda_l A_{l,i}\right) + \sum_{l=1}^{12} \frac{\lambda_l A_{l,i}}{\sum_{k=1}^{12} \lambda_k A_{k,i}} w_i \right] - \sum_{l=1}^{12} \lambda_l c_l \\ &= \text{const} + \sum_{i=1}^7 \left[w_i \ln\left(\sum_{l=1}^{12} \lambda_l A_{l,i}\right) + w_i \right] - \sum_{l=1}^{12} \lambda_l c_l \\ &= \text{const} + \sum_{i=1}^7 w_i \ln\left(\sum_{l=1}^{12} \lambda_l A_{l,i}\right) - \sum_{l=1}^{12} \lambda_l c_l \end{aligned}$$

And then the goal of the dual is:

$$\begin{aligned} \max_{\lambda} g(\lambda) &= \max_{\lambda} \sum_{i=1}^7 w_i \ln\left(\sum_{l=1}^{12} \lambda_l A_{l,i}\right) - \sum_{l=1}^{12} \lambda_l c_l \\ \text{subject to } \lambda_l &\geq 0 \text{ for } l = 1, \dots, 12 \end{aligned}$$

Then, the dual gradient ascent depends on the gradient of $g(\lambda)$, which, for every l , is:

$$\frac{\partial g(\lambda)}{\partial \lambda_l} = \sum_{i=1}^7 w_i \frac{A_{l,i}}{\sum_{k=1}^{12} \lambda_k A_{k,i}} - c_l$$

and the full gradient is:

$$\nabla g(\lambda) = \begin{bmatrix} \frac{\partial g(\lambda)}{\partial \lambda_1} \\ \frac{\partial g(\lambda)}{\partial \lambda_2} \\ \vdots \\ \frac{\partial g(\lambda)}{\partial \lambda_{12}} \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^7 w_i \frac{A_{1,i}}{\sum_{k=1}^{12} \lambda_k A_{k,i}} - c_1 \\ \sum_{i=1}^7 w_i \frac{A_{2,i}}{\sum_{k=1}^{12} \lambda_k A_{k,i}} - c_2 \\ \vdots \\ \sum_{i=1}^7 w_i \frac{A_{12,i}}{\sum_{k=1}^{12} \lambda_k A_{k,i}} - c_{12} \end{bmatrix}$$

And finally, the dual gradient ascent algorithm is:

$$\lambda^{(t+1)} = \left[\lambda^{(t)} + \gamma \nabla g(\lambda^{(t)}) \right]^+$$

where $[\cdot]^+$ is the projection onto non-negative values (i.e. $[\lambda]^+ = \max(\lambda, 0)$).

To get the optimal λ^* , repeat the above until convergence, then at the end, $x^* = \frac{w_i}{\sum_{l=1}^{12} \lambda_l^* A_{l,i}}$.

(b)

The code below implements the above algorithm.

In [176...

```
import numpy as np
import matplotlib.pyplot as plt

# Parameters
lambda_ = np.random.uniform(0, 10, 12)
lr = 0.01
n_iter = 10000

# Constants
A = np.zeros((12, 7))
A[2, 0] = 1 # A_{3,1} = 1
A[8, 0] = 1 # A_{9,1} = 1
A[3, 1] = 1 # A_{4,2} = 1
A[8, 1] = 1 # A_{9,2} = 1
A[2, 2] = 1 # A_{3,3} = 1
A[9, 2] = 1 # A_{10,3} = 1
A[3, 3] = 1 # A_{4,4} = 1
A[9, 3] = 1 # A_{10,4} = 1
A[4, 4] = 1 # A_{5,5} = 1
A[7, 4] = 1 # A_{8,5} = 1
A[4, 5] = 1 # A_{5,6} = 1
A[8, 5] = 1 # A_{9,6} = 1
A[0, 6] = 1 # A_{1,7} = 1
A[5, 6] = 1 # A_{6,7} = 1
A[10, 6] = 1 # A_{11,7} = 1

w = np.array([1, 1, 1, 1, 2, 2, 2])
c = np.array([1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2])

# Helper function with epsilon to avoid division by zero
def _sum_lambda_A(lambda_, i):
    return np.sum(lambda_ * A[:, i]) + 1e-10

# Corrected gradient of g
def grad_g(lambda_):
    grad_g = np.zeros(12)
    for l in range(12):
        grad_g[l] = np.sum([w[i] * A[l, i] / _sum_lambda_A(lambda_, i) for i in range(7)]) - c[l]
    return grad_g

# Calculate x
def calc_x(lambda_):
    x = np.array([w[i] / _sum_lambda_A(lambda_, i) for i in range(7)])
    return x

# Project onto non-negative orthant
def project_lambda(lambda_):
    return np.clip(lambda_, 0, None)

# Projected gradient ascent
def projected_dual_gradient_descent(lambda_0, lr, n_iter):
    lambda_t = np.array(lambda_0, dtype=float)
    x_history = []
    lambda_history = [lambda_0.copy()]

    for _ in range(n_iter):
```

```

    x = calc_x(lambda_t)
    x_history.append(x.copy())
    grad = grad_g(lambda_t)
    lambda_t = lambda_t + lr * grad # Ascent for maximization
    lambda_t = project_lambda(lambda_t)
    lambda_history.append(lambda_t.copy())
    x_history.append(calc_x(lambda_t))
    return np.array(lambda_history), np.array(x_history)

# Run the algorithm
lambda_history, x_history = projected_dual_gradient_descent(lambda_, lr, n_iter)
print(f"Final x: {x_history[-1]}")
print(f"Final lambda: {lambda_history[-1]}")
print("Constraint Ax <= c:", np.all(A @ x_history[-1] <= c + 1e-6))

# Plot x evolution
T = np.arange(n_iter + 1)
for i in range(7):
    plt.plot(T, x_history[:, i], label=f"$x_{{{i+1}}}$")
plt.xlabel("Iteration t")
plt.ylabel("$x_i$")
plt.legend()
plt.grid(True)
plt.title("Evolution of x(t)")
plt.tight_layout()
plt.show()
plt.close()

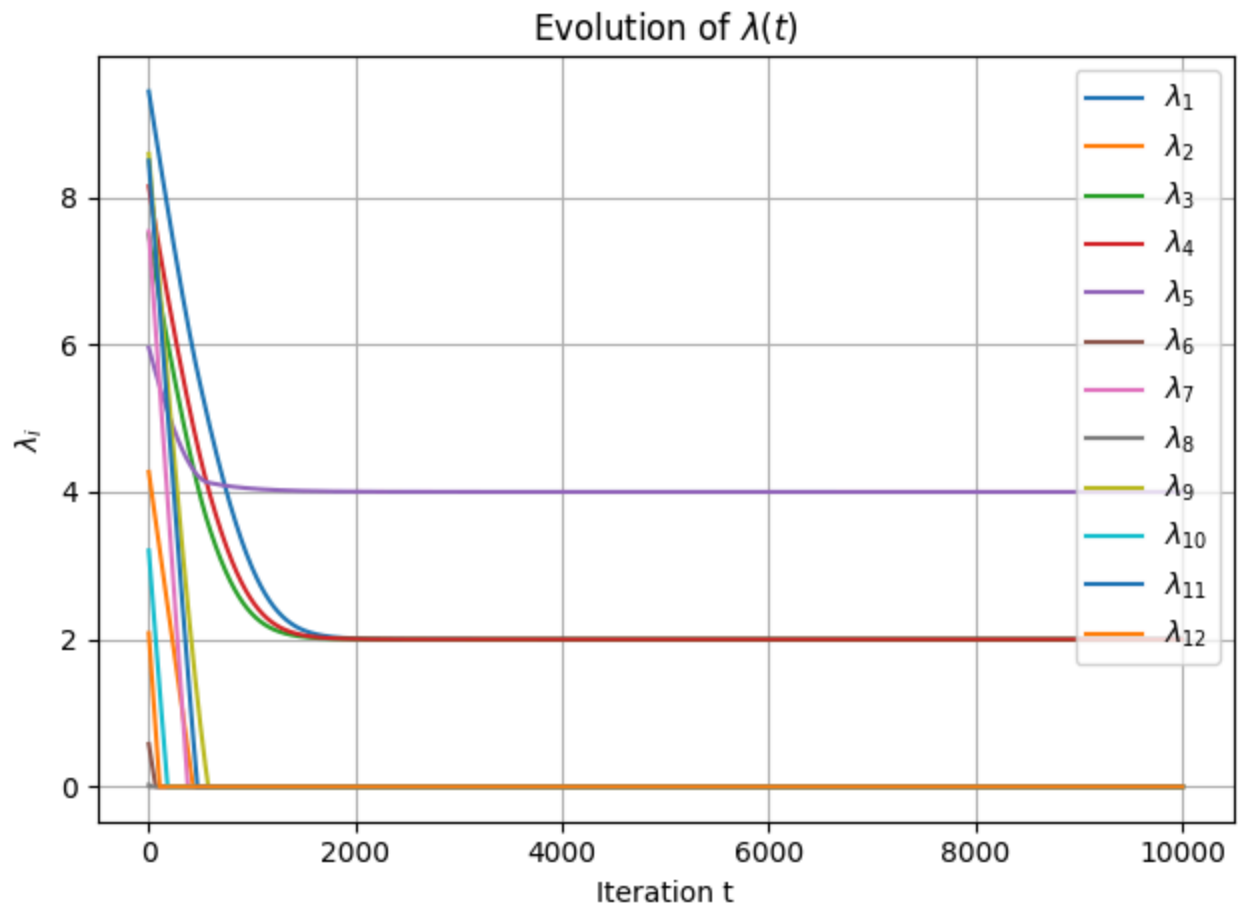
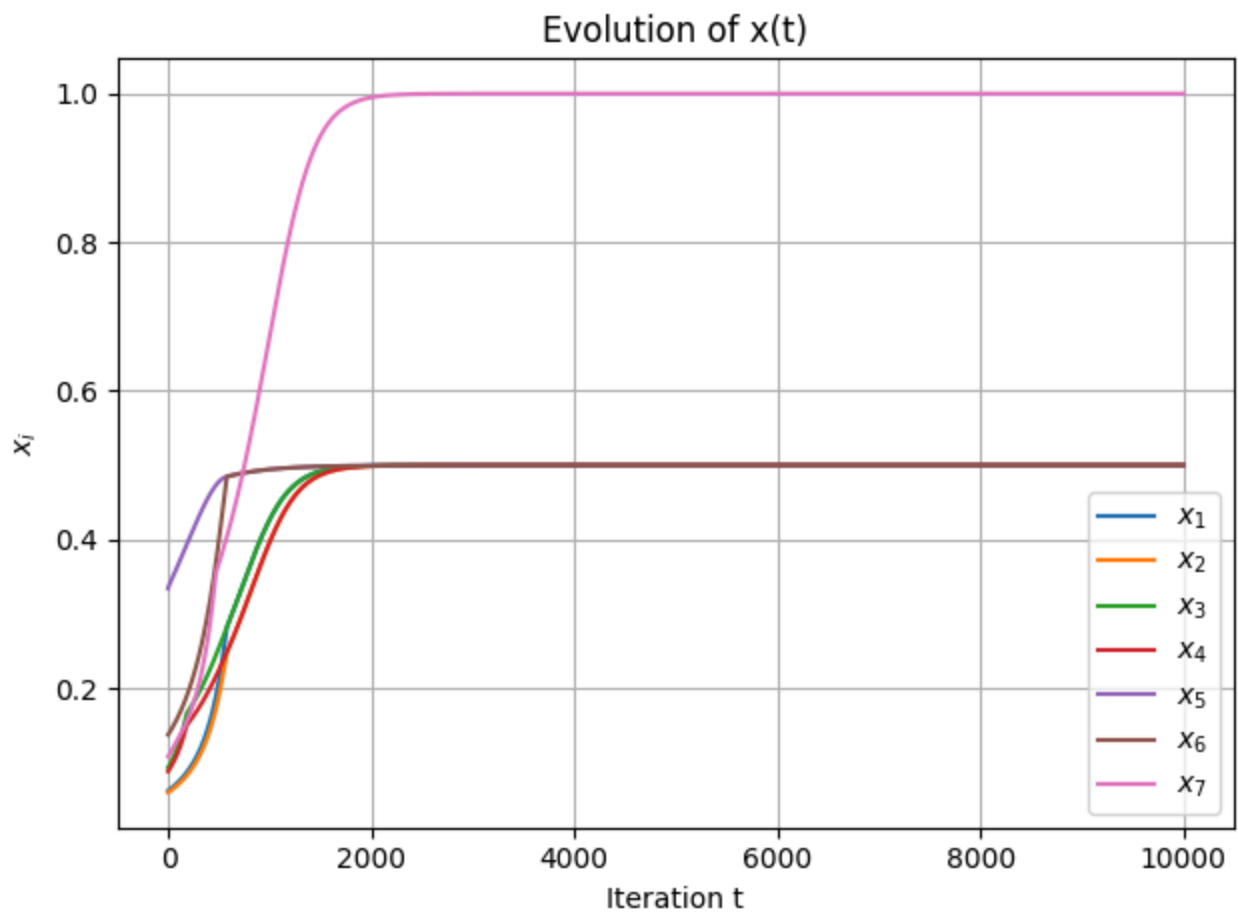
# Plot lambda evolution
T = np.arange(n_iter + 1)
for i in range(12):
    plt.plot(T, lambda_history[:, i], label=f"$\\lambda_{{{i+1}}}$")
plt.xlabel("Iteration t")
plt.ylabel("$\\lambda_i$")
plt.legend()
plt.grid(True)
plt.title("Evolution of $\\lambda(t)$")
plt.tight_layout()
plt.show()
plt.close()

```

```

Final x: [0.5 0.5 0.5 0.5 0.5 0.5 1. ]
Final lambda: [2. 0. 2. 2. 4. 0. 0. 0. 0. 0. 0. 0.]
Constraint Ax <= c: True

```



(c)

Since this is a convex optimization problem, the KKT conditions are necessary and sufficient to establish optimality. Below, the function checks if the KKT conditions hold for the solution above.

In [177...

```
def check_kkt_conditions(x, lambda_, eps=1e-6):
    # Primal feasibility failed
    if np.any(x < 0):
        return False

    # Primal feasibility failed for resource constraints
    Ax = A @ x
    if np.any(Ax > c + eps):
        return False

    # Dual feasibility failed
    if np.any(lambda_ < 0):
        return False

    # Complementary slackness failed
    slack = Ax - c
    for l in range(len(c)):
        product = lambda_[l] * slack[l]
        if abs(product) > eps:
            return False

    # Stationarity condition failed
    for i in range(len(x)):
        grad_L_x_i = w[i] / x[i] - np.sum(lambda_ * A[:, i])
        if abs(grad_L_x_i) > eps:
            return False
    return True

# Verify KKT conditions
if_kkt_conditions_hold = check_kkt_conditions(x_history[-1], lambda_history[-1])
if if_kkt_conditions_hold:
    print("KKT conditions hold, the solution is optimal.")
else:
    print("KKT conditions do not hold, the solution is not optimal.")
```

KKT conditions hold, the solution is optimal.

As shown above, since the KKT conditions hold, the solution is optimal.